Dynamic Programming

Model-Based Planning — Lecture 2/4

Etienne Chassaing

CentraleSupélec

Plan

1. Policy Evaluation
2. Policy Iteration
3. Value Iteration
4. Convergence and Contraction

Where we are

Lecture 1 — MDPs & Bellman

- Formalized MDP as (S, A, p, r, γ)
- Bellman equations: $v_\pi(s)$, $q_\pi(s, a)$
- Optimal equations: $v_*(s)$, $q_*(s, a)$

Lecture 2 — Dynamic Programming

- We know the model $p(s'|s, a)$
- We know $r(s, a)$
- **Goal:** Compute v_* or π_* exactly

Key insight: Bellman equations are systems of linear equations — we can solve them iteratively by repeatedly applying the Bellman operator until convergence.

Plan

1. Policy Evaluation
2. Policy Iteration
3. Value Iteration
4. Convergence and Contraction

Policy Evaluation

Goal: Given a policy π, compute $v_\pi(s)$ for all states s.

Method: Iteratively solve the Bellman expectation equation:

$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r|s, a)[r + \gamma v_\pi(s')]$$

Algorithm:

1. Initialize $v(s) = 0$ for all s
2. Repeat (policy evaluation step):
   - For each state s, update:

$$v_{\text{new}}(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)[r + \gamma v(s')]$$

3. Until convergence: $\|v_{\text{new}} - v\| < \epsilon$

Policy Evaluation — Example

Gridworld example: Given $\pi(\text{up}|s) = \pi(\text{left}|s) = 0.5$, compute $v^\pi(s)$.

Iteration 0:

- $v(s) = 0$ for all s

Iteration 1:

- Compute $v_{\text{new}}(s)$ using $v = 0$
- Update values based on immediate rewards

Iteration k: Values gradually converge to the true v^π

Key: Each iteration uses the **most recent value estimates**, creating a bootstrapping effect.

Plan

Policy Iteration

Observation: We can **improve** a policy using its value function:

$$\pi'(s) = \arg\max_a \sum_{s',r} p(s', r|s, a)[r + \gamma v^\pi(s')]$$

Policy Iteration = Evaluation + Improvement:

1. **Policy Evaluation:** Compute v^π (solve Bellman equation)
2. **Policy Improvement:** Greedify with respect to v^π
3. Repeat until policy converges: $\pi' = \pi$

Convergence: Policy iteration converges to the optimal policy π_* in a finite number of iterations (each iteration either improves or stabilizes the policy).

Policy Iteration — Pseudocode

Policy Iteration Algorithm

1. Initialize policy π arbitrarily (e.g., random)
2. **repeat**:
   - 2.1 **Policy Evaluation:** Compute v^π via iterative Bellman updates
   - 2.2 **Policy Improvement:** Check if policy changed
     - For each s: $\pi_{\text{old}}(s) \leftarrow \pi(s)$
     - For each s: $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r|s, a)[r + \gamma v(s')]$
   - 2.3 If $\pi = \pi_{\text{old}}$ (policy stable): **break**

Each "repeat" includes a full policy evaluation (itself iterative) — can be slow if evaluation takes many sweeps.

Plan

1. Policy Evaluation
2. Policy Iteration
3. Value Iteration
4. Convergence and Contraction

Value Iteration

Idea: Skip full policy evaluation. Instead, just apply the **Bellman optimality operator**:

$$v(s) \leftarrow \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma v(s')]$$

Value Iteration = Convergence without explicit policy:

1. Initialize $v(s) = 0$ for all s
2. **repeat** (value update):
   - For each state s:
   $$v_{\text{new}}(s) \leftarrow \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma v(s')]$$
3. Until $\|v_{\text{new}} - v\| < \epsilon$
4. Extract policy: $\pi_*(s) = \arg\max_a \sum_{s',r} p(s', r|s, a)[r + \gamma v_*(s')]$

Pro: One update rule. **Con:** May take more iterations than policy iteration.

Policy Iteration vs Value Iteration

	Policy Iteration	Value Iteration
Convergence guarantee	Yes (π_* in finite steps)	Yes (v_* asymptotically)
Speed (typical)	Fewer outer iterations	More iterations but simpler update
Computation per step	One full PE sweep	One Bellman max
When to use	Medium-sized MDPs	Large state spaces

In practice: Most modern RL uses value iteration or variants (TD learning, etc.)

Plan

1. Policy Evaluation
2. Policy Iteration
3. Value Iteration
4. Convergence and Contraction

The Bellman Operator as a Contraction

Why do these algorithms converge?

Define the **Bellman optimality operator**:

$$(\mathcal{T}^* v)(s) = \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma v(s')]$$

Theorem (Contraction Mapping): For discount factor $\gamma < 1$, $\mathcal{T}^*$ is a γ-contraction in sup-norm:

$$\|\mathcal{T}^* v_1 - \mathcal{T}^* v_2\|_\infty \leq \gamma \|v_1 - v_2\|_\infty$$

Consequence (Banach Fixed-Point Theorem):

- $\mathcal{T}^*$ has a unique fixed point v_*
- Repeated application: $v_k = \mathcal{T}^* v_{k-1}$ converges to v_*
- Convergence rate: $\|v_k - v_*\| \leq \gamma^k \|v_0 - v_*\|$ (exponential)

Why $\gamma < 1$ Matters

With $\gamma < 1$:

- Bellman operator is a **contraction**
- Iterations provably converge
- Error shrinks exponentially
- Mathematical guarantee

With $\gamma = 1$ (undiscounted):

- May diverge (no contraction)
- Works *only* for episodic tasks (finite horizon)
- Or with bounded rewards + finite state space

Rule of thumb: Always use $\gamma < 1$ for infinite-horizon problems. Set $\gamma \approx 0.99$ by default.

Key Takeaways

- **Dynamic Programming** solves MDPs when the model $p(s'|s, a)$ is known
- **Policy Evaluation:** Solve for v^π given π
- **Policy Iteration:** Repeat evaluation + greedy improvement → converges to π_*
- **Value Iteration:** Apply Bellman max directly → simpler, converges to v_*
- **Convergence guarantee:** Bellman operator is a γ-contraction ($\gamma < 1$)
- **Next:** Model-free learning (no need to know p) — Monte Carlo, TD, Q-learning

Questions?

Your feedback on this lecture is welcome

etienne.chassaing@centralesupelec.fr

Survey form (anonymous feedback):
`forms.gle/your-form-link`

Next: Kahoot quiz at start of Lecture 3!